Reporte de Práctica

Comunicación I2C entre 4 Arduinos

Sistemas Programables | **Jonathan Alexis Morales Ramirez** | Septiembre 2026

1. Introducción

El protocolo I2C (Inter-Integrated Circuit) es un estándar de comunicación serial síncrona que permite conectar múltiples dispositivos usando solo dos líneas: SDA (datos) y SCL (reloj), más una tierra común. Cada dispositivo esclavo se identifica mediante una dirección única de 7 bits, lo que permite que el maestro seleccione con quién comunicarse en cada momento.
En esta práctica se implementó un bus I2C con cuatro Arduino UNO R4 WiFi: un maestro y tres esclavos con funciones distintas, demostrando la comunicación bidireccional y el control de múltiples periféricos sobre el mismo bus.

2. Objetivos

- Comprender el funcionamiento del bus I2C y la asignación de direcciones.
- Implementar la comunicación entre un maestro y tres esclavos con distintas funciones.
- Controlar un LED, un servomotor y leer un potenciómetro a través del bus I2C.
- Usar millis() en el maestro para una temporización no bloqueante.
- Identificar y corregir errores frecuentes en circuitos I2C.

3. Materiales utilizados

Componente	Descripción	Cantidad
Arduino UNO R4 WiFi	Microcontrolador (1 maestro y 3 esclavos)	4
Protoboard	Plataforma de prototipado	1
LED	Controlado por Esclavo 1 (dir. 0x08)	1
Resistencia 470 Ω	Limitadora de corriente para LED (R4)	1
Micro servo	Controlado por Esclavo 2 (dir. 0x09)	1

Componente	Descripción	Cantidad
Potenciómetro	Leído por Esclavo 3 (dir. 0x0A)	1
Resistencias 4.7 kΩ	Pull-up en líneas SDA y SCL	2
Cables Dupont	Conexiones entre placas y protoboard	Varios

4. Descripción del sistema

El maestro controla toda la comunicación del bus. Cada 500 ms solicita el valor del potenciómetro al Esclavo 3, convierte ese valor (0-1023) a un ángulo (0° - 180°) y lo envía al Esclavo 2 para mover el servomotor. Adicionalmente, el usuario puede escribir '1' o '0' en el Monitor Serie para encender o apagar el LED del Esclavo 1.

Arduino	Dirección I2C	Función
Maestro	—	Controla toda la comunicación del bus
Esclavo 1	0x08	Enciende o apaga un LED según orden del maestro
Esclavo 2	0x09	Mueve el servomotor al ángulo recibido
Esclavo 3	0x0A	Lee el potenciómetro y envía el valor al maestro

5. Metodología

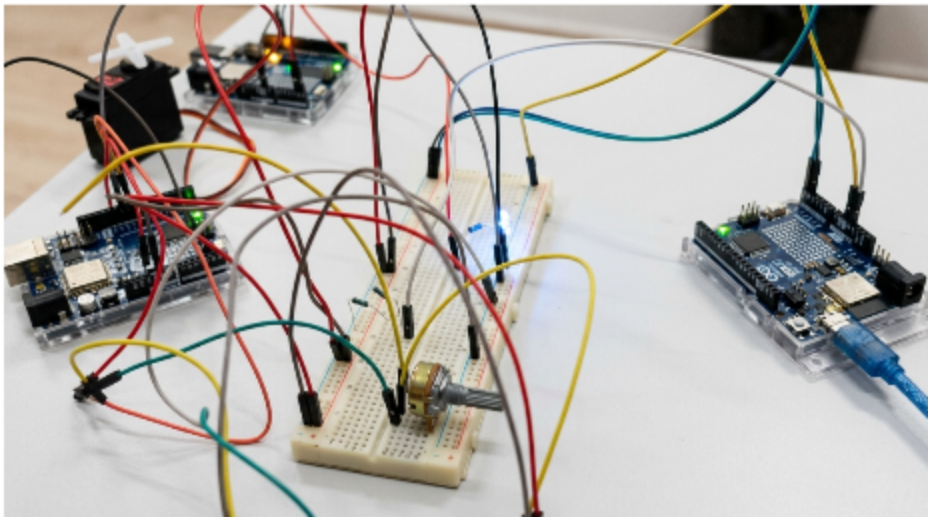
Paso 1: Armado del circuito
Se conectaron los cuatro Arduinos compartiendo las líneas SDA (A4) y SCL (A5), con tierra común entre todos. Se agregaron resistencias de 4.7 kΩ como pull-up en SDA y SCL. Cada placa se alimentó por su propio cable USB.
Paso 2: Programación del maestro
Se configuró Wire.begin() sin dirección (modo maestro). Se usó millis() para consultar el potenciómetro cada 500 ms sin bloquear la lectura del Monitor Serie. Se agregó while(!Serial && millis()<3000) para el puerto USB nativo del R4.
Paso 3: Programación de los esclavos
Cada esclavo se inicializó con Wire.begin(dirección). El Esclavo 1 usa onReceive() para leer la

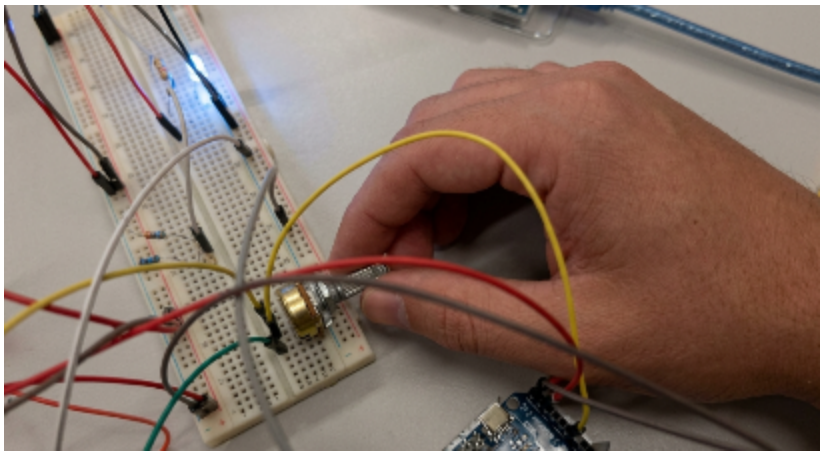
orden del LED. El Esclavo 2 usa onReceive() y mueve el servo en el loop() con una bandera para no operar dentro de la ISR. El Esclavo 3 usa onRequest() para enviar el valor del potenciómetro en 2 bytes.

Paso 4: Pruebas y verificación

Se verificó el control del LED desde el Monitor Serie con '1' y '0', el movimiento del servo al girar el potenciómetro, y la detección de esclavos no conectados mediante endTransmission().

6. Evidencia del circuito





7. Errores frecuentes identificados

- **Resistencia con ambas patas en la misma columna:** Los huecos de una columna están unidos internamente, por lo que la corriente pasa sin resistencia y el LED puede quemarse. Las patas deben ir en columnas distintas.
- **Leer varias veces el mismo dato de I2C:** El dato se consume en la primera lectura; las siguientes devuelven un valor inválido. Se debe leer una sola vez y guardar en una variable.
- **Operar el servo dentro de la ISR:** La librería Servo usa interrupciones internas que pueden entrar en conflicto. La solución es usar una bandera y mover el servo en el loop().

8. Conclusiones

- El bus I2C permite comunicar varios dispositivos usando solo dos líneas (SDA y SCL) más tierra común. Agregar un esclavo no requiere pines adicionales, solo una dirección diferente.
- Cada esclavo debe tener una dirección única. Si dos comparten la misma, los datos se corrompen.
- El maestro controla toda la comunicación: decide con quién habla, cuándo y si pide o manda datos.
- El uso de millis() en el maestro permite atender el Monitor Serie y consultar el potenciómetro sin bloquearse.
- La verificación de endTransmission() permite detectar cuando un esclavo no contesta (NACK) en lugar de que el sistema se congele.